

# WI4014TU – Numerical Analysis for PDE's

## Lecture 11

Numerical time integration, Forward- and Backward-Euler methods, convergence, stability, solution of the diffusion equation

# Time-dependent problems

Diffusion equation:

$$\frac{\partial u}{\partial t} - \nabla \cdot (k \nabla u) = f$$

Wave equation:

$$\frac{1}{c^2} \frac{\partial^2 u}{\partial t^2} - \Delta u = f$$

Unknown function  $u(x, y, t) \Rightarrow$  grid values  $u(x_i, y_j, t)$  stored in a vector  $\mathbf{u}(t) \Rightarrow$  solution vector changes in time. How to compute the time evolution of  $\mathbf{u}(t)$ ?

## Numerical time integration

# First-order time evolution

Let  $\mathbf{u}(t) \in \mathbb{R}^n$  be a time-evolving vector and  $\mathbf{f}(t, \mathbf{u}) \in \mathbb{R}^n$  a vector function depending on both time  $t$  and the vector  $\mathbf{u}$ . Consider a system of the first-order ODE's

$$\frac{d\mathbf{u}}{dt} = \mathbf{f}(t, \mathbf{u}), \quad t \geq t_0, \quad \mathbf{u}(t_0) = \mathbf{u}_0$$

$$\frac{d}{dt} \begin{bmatrix} u_1 \\ u_2 \\ \vdots \end{bmatrix} = \begin{bmatrix} f_1(t, u_1, u_2, \dots) \\ f_2(t, u_1, u_2, \dots) \\ \vdots \end{bmatrix}$$

For example, a linear system of ODE's

$$\mathbf{u}' = -A\mathbf{u} + \mathbf{s},$$

where  $A$  is some matrix. But it can also be a nonlinear function  $\mathbf{f}$ .

## Prediction by time integration

We know  $\mathbf{u}(t_0)$  – initial condition. Suppose we want to compute  $\mathbf{u}(t_0 + h)$ , for some  $h > 0$ . Integrating the ODE from  $t_0$  to  $t_0 + h$ , and using the Fundamental Theorem of Calculus (FTC), we get

$$\int_{t_0}^{t_0+h} \mathbf{u}' dt = \int_{t_0}^{t_0+h} \mathbf{f}(t, \mathbf{u}(t)) dt$$

$$\mathbf{u}(t_0 + h) - \mathbf{u}(t_0) = \int_{t_0}^{t_0+h} \mathbf{f}(t, \mathbf{u}(t)) dt$$

$$\mathbf{u}(t_0 + h) = \mathbf{u}(t_0) + \int_{t_0}^{t_0+h} \mathbf{f}(t, \mathbf{u}(t)) dt$$

# Numerical approximation

Hence, the problem of finding  $\mathbf{u}(t_0 + h)$  reduces to the problem of finding a good numerical approximation (quadrature) for the integral of  $\mathbf{f}(t, \mathbf{u})$ :

$$\int_{t_0}^{t_0+h} \mathbf{f}(t, \mathbf{u}(t)) dt \approx \sum_{k=1}^K w_k \mathbf{f}(t_k, \mathbf{u}(t_k)), \quad t_0 \leq t_k \leq t_0 + h$$

Having found  $\mathbf{u}(t_0 + h)$  we can apply the same procedure to find  $\mathbf{u}(t_0 + h + h)$ , etc. – time-iterations (recurrence).

# Forward-Euler (Euler's Forward) method



Single-point approximation of the integral at the starting point:

$$\int_{t_0}^{t_0+h} \mathbf{f}(t, \mathbf{u}(t)) dt \approx h \mathbf{f}(t_0, \mathbf{u}(t_0))$$

Explicit one-step prediction:

$$\mathbf{u}(t_0 + h) \approx \mathbf{u}(t_0) + h \mathbf{f}(t_0, \mathbf{u}(t_0))$$

Forward-Euler (FE) iterations:

$$\mathbf{u}(t_k + h) \approx \mathbf{u}(t_k) + h \mathbf{f}(t_k, \mathbf{u}(t_k)), \quad k = 0, 1, \dots$$

# Backward-Euler (Euler's Backward) method

Single-point approximation of the integral at the end point:

$$\int_{t_k}^{t_{k+1}} \mathbf{f}(t, \mathbf{u}(t)) dt \approx h \mathbf{f}(t_{k+1}, \mathbf{u}(t_{k+1}))$$

Implicit BE iterations

$$\mathbf{u}(t_{k+1}) \approx \mathbf{u}(t_k) + h \mathbf{f}(t_{k+1}, \mathbf{u}(t_{k+1})), \quad k = 0, 1, \dots$$

- This requires solving a vector equation to find  $\mathbf{u}(t_{k+1})$ . The Backward-Euler is an *implicit* method, whereas, the Forward-Euler is an *explicit* method



# Trapezoidal method

Two-point approximation of the integral:

$$\int_{t_k}^{t_{k+1}} \mathbf{f}(t, \mathbf{u}(t)) dt \approx \frac{1}{2}h[\mathbf{f}(t_k, \mathbf{u}(t_k)) + \mathbf{f}(t_{k+1}, \mathbf{u}(t_{k+1}))]$$

Implicit trapezoidal iterations

$$\mathbf{u}(t_{k+1}) \approx \mathbf{u}(t_k) + \frac{1}{2}h[\mathbf{f}(t_k, \mathbf{u}(t_k)) + \mathbf{f}(t_{k+1}, \mathbf{u}(t_{k+1}))], \quad k = 0, 1, \dots$$

## Other integration methods

$$\int_{t_0}^{t_0+h} \mathbf{f}(t, \mathbf{u}(t)) dt \approx \sum_{i=1}^k w_i \mathbf{f}(t_k, \mathbf{u}(t_k)), \quad t_0 \leq t_k \leq t_0 + h$$

- Multistep methods: Adams-Bashforth, backward differentiation formulae (BDF)
- Explicit and implicit Runge-Kutta methods (ERK, IRK)



## Convergence and order of time integration methods

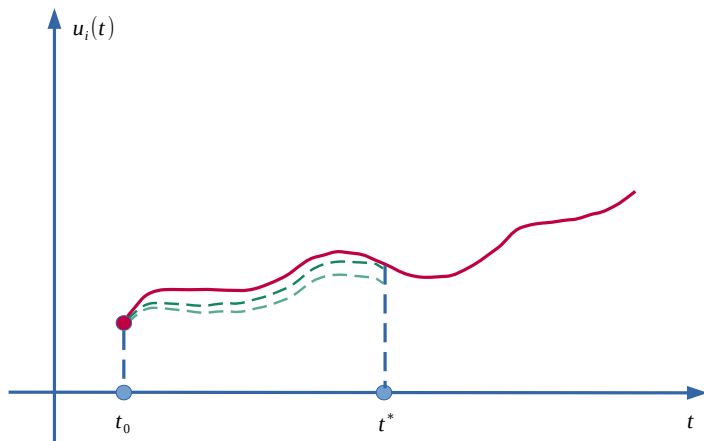
# Convergence of time integration methods

- Suppose we have applied some time integration method on the interval  $t \in [t_0, t^*]$  with the time step  $h = (t^* - t_0)/K$ , and obtained the numerical approximation  $\mathbf{u}_h(t_k)$  of the actual  $\mathbf{u}(t_k)$ , for all  $k = 0, 1, \dots, K$ .
- We say that the method is *convergent* if for every  $\mathbf{f}(t, \mathbf{u})$  and every  $t^* > t_0$  it is true that

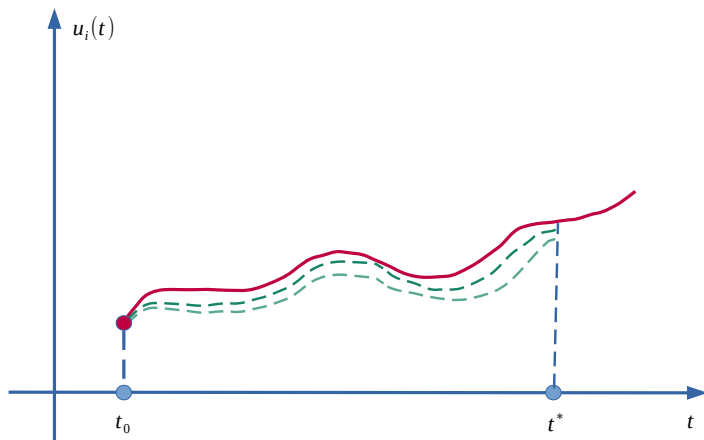
$$\lim_{h \rightarrow 0^+} \max_{k=0,1,\dots,K} \|\mathbf{u}_h(t_k) - \mathbf{u}(t_k)\| = 0$$

Global max-norm converges to zero as time step  $h$  decreases for any finite  $t^*$ .

# Idea of convergence



# Idea of convergence



# Convergence and global error of the Forward-Euler method

**Theorem:** Let  $\mathbf{u}(t)$  be continuously differentiable and  $\mathbf{f}(t, \mathbf{u})$  a *Lipschitz* function, i.e., there is  $\lambda > 0$  such that  $\|\mathbf{f}(t, \mathbf{u}) - \mathbf{f}(t, \mathbf{v})\| \leq \lambda \|\mathbf{u} - \mathbf{v}\|$  for all  $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ . Then, the Forward-Euler method is convergent.

**Proof:** Denoting  $\mathbf{e}_h(t_k) = \mathbf{u}_h(t_k) - \mathbf{u}(t_k)$

$$\mathbf{u}(t_{k+1}) = \mathbf{u}(t_k) + h\mathbf{u}'(t_k) + \mathcal{O}(h^2) = \mathbf{u}(t_k) + h\mathbf{f}(t_k, \mathbf{u}(t_k)) + \mathcal{O}(h^2)$$

$$\mathbf{e}_h(t_{k+1}) = \mathbf{e}_h(t_k) + h[\mathbf{f}(t_k, \mathbf{u}(t_k) + \mathbf{e}_h(t_k)) - \mathbf{f}(t_k, \mathbf{u}(t_k))] + \mathcal{O}(h^2)$$

$$\|\mathbf{e}_h(t_{k+1})\| \leq \|\mathbf{e}_h(t_k)\| + h\|\mathbf{f}(t_k, \mathbf{u}(t_k) + \mathbf{e}_h(t_k)) - \mathbf{f}(t_k, \mathbf{u}(t_k))\| + Ch^2$$

$$\|\mathbf{e}_h(t_{k+1})\| \leq (1 + h\lambda)\|\mathbf{e}_h(t_k)\| + Ch^2, \quad k = 0, 1, \dots, n$$

$$\|\mathbf{e}_h(t_k)\| \leq \frac{C}{\lambda} h [(1 + h\lambda)^k - 1] < \frac{C}{\lambda} h (e^{kh\lambda} - 1) \leq \frac{C}{\lambda} (e^{\lambda t^*} - 1) h$$

$$\lim_{h \rightarrow 0^+} \max_k \|\mathbf{e}_h(t_k)\| \leq \frac{C}{\lambda} (e^{\lambda t^*} - 1) \lim_{h \rightarrow 0^+} h = 0$$

# Local and global orders of time integration methods

- We say that the time integration method is *locally of order  $p$*  if

$$\mathbf{u}_h(t_1) - \mathbf{u}(t_1) = \mathcal{O}(h^p)$$

- Method of local order  $p$  recovers exactly any polynomial of degree  $p - 1$  or less
- Forward Euler method is locally  $\mathcal{O}(h^2)$ :

$$\begin{aligned}\mathbf{u}_h(t_1) - \mathbf{u}(t_1) &= \mathbf{u}_h(t_0) + h\mathbf{f}(t_0, \mathbf{u}_h(t_0)) + \mathcal{O}(h^2) \\ &\quad - \mathbf{u}(t_0) + h\mathbf{f}(t_0, \mathbf{u}(t_0)) - \mathcal{O}(h^2) = \mathcal{O}(h^2)\end{aligned}$$

since  $\mathbf{u}_h(t_0) = \mathbf{u}(t_0)$ . *Global* error typically drops to order  $p - 1$ , and convergence analysis shows that the global error is, indeed, proportional to  $h$ .



# Convergence and order of other methods

- Backward Euler method is convergent and locally  $\mathcal{O}(h^2)$ , globally  $\mathcal{O}(h)$ .
- Trapezoidal rule is an implicit, convergent, locally  $\mathcal{O}(h^3)$  and globally  $\mathcal{O}(h^2)$  method

$$\|\mathbf{e}_h(t_k)\| \leq \frac{Ch^2}{\lambda} \exp\left(\frac{\lambda t^*}{1 - \lambda h/2}\right), \quad k = 0, 1, \dots, K$$

## Method of lines for diffusion equation

Method of lines discretizes in space, but keeps time continuous. Serves as a theoretical benchmark in stability analysis of numerical time-integration methods.

## With FDM/FVM spatial discretization

Later we shall see that after FDM discretization in space:

$-\Delta u(\mathbf{x}, t) \rightarrow A\mathbf{u}(t)$ ,  $\mathbf{u}(t) \in \mathbb{R}^n$ ,  $A \in \mathbb{R}^{n \times n}$ , the diffusion equation becomes the system of ODE's:

$$\mathbf{u}' = -A\mathbf{u}, \quad \mathbf{u}(0) = \mathbf{u}_0$$

Assume  $A = V\Lambda V^{-1}$  exists. Then,  $\mathbf{u}(t) = V\mathbf{b}(t)$ , and  $\mathbf{b}(t) = V^{-1}\mathbf{u}(t)$ .

$$V\mathbf{b}' = -V\Lambda V^{-1}\mathbf{u}, \quad \mathbf{u}_0 = V\mathbf{b}_0$$

$$\mathbf{b}' = -\Lambda\mathbf{b}, \quad \mathbf{b}(0) = V^{-1}\mathbf{u}_0,$$

$$b_j'(t) = -\lambda_j b_j(t), \quad j = 1, \dots, n$$

$$b_j(t) = b_j(0)e^{-\lambda_j t}, \quad j = 1, \dots, n$$

$$\mathbf{b}(t) = e^{-\Lambda t}\mathbf{b}(0) = e^{-\Lambda t}V^{-1}\mathbf{u}(0)$$

$$\mathbf{u}(t) = Ve^{-\Lambda t}V^{-1}\mathbf{u}(0)$$

This is the exact solution of the discrete time-evolution problem.

## Case of (negative) FD Laplacian

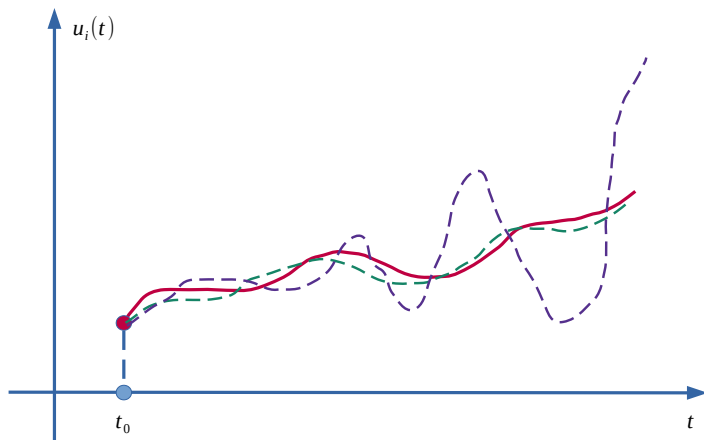
For  $-\Delta \rightarrow A$  with the FDM method the eigenvalues are positive  $\lambda_j > 0$ ,  $j = 1, \dots, n$ . Writing out  $\mathbf{u}(t) = V\mathbf{b}(t)$  and using the method-of-lines to find  $\mathbf{u}(t)$ , we get

$$\mathbf{u}(t) = \sum_{j=1}^n b_j(t) \mathbf{v}_j$$
$$b_j(t) = b_j(0) e^{-\lambda_j t}$$

Shows that the amplitudes  $b_j(t)$  of all expansion terms  $b_j(t) \mathbf{v}_j$  of the solution decay exponentially in time.

## Stability of time integration methods

# Idea of stability



# Stability of the forward Euler method

After FDM/FVM discretization the diffusion equation becomes

$$\mathbf{u}' = -A\mathbf{u}, \quad \mathbf{u}(t_0) = \mathbf{u}_0$$

the Forward-Euler method with the time step  $h$  produces iterations  $k = 0, 1, \dots$ :

$$\mathbf{u}_h(t_k) = \mathbf{u}_h(t_{k-1}) - hA\mathbf{u}_h(t_{k-1}) = [I - hA]\mathbf{u}_h(t_{k-1}) = [I - hA]^k \mathbf{u}_0$$

Let  $A$  be diagonalizable:  $A = V\Lambda V^{-1}$  with eigenvalues  $\lambda_j$ ,  $j = 1, \dots, n$ .

$$\mathbf{u}(t_k) = V e^{\Lambda t_k} V^{-1} \mathbf{u}_0 = \sum_{j=1}^n b_j(0) e^{-\lambda_j t_k} \mathbf{v}_j \quad (\text{method of lines})$$

$$\mathbf{u}_h(t_k) = [I - hA]^k \mathbf{u}_0 = V [I - h\Lambda]^k V^{-1} \mathbf{u}_0 = \sum_{j=1}^n b_j(0) (1 - h\lambda_j)^k \mathbf{v}_j$$

Hence, for  $\lambda_j > 0$ ,  $e^{-\lambda_j t_k}$  always decays with  $t_k$ , but  $|1 - h\lambda_j|^k$  may be growing with  $k$  if  $h$  is too large. The Forward-Euler method is not always stable.

# Conditional stability of the forward Euler method

Deriving the condition for the stability of the Forward-Euler method:

$$\begin{aligned} |1 - h\lambda_j| &< 1 \\ -1 &< 1 - h\lambda_j < 1 \\ -2 &< -h\lambda_j < 0 \\ 0 &< h\lambda_j < 2 \\ 0 &< h < \frac{2}{\lambda_j}, \quad j = 1, \dots, n \end{aligned}$$

It is common to denote the time step  $h$  as  $h_t$  or  $\Delta t$ :

$$0 < \Delta t < \frac{2}{\max_j \lambda_j}$$



# Unconditional stability of Backward-Euler method

Backward-Euler method with  $\Delta t = h$  produces iterations  $k = 0, 1, \dots$ :

$$\mathbf{u}_h(t_k) = \mathbf{u}_h(t_{k-1}) - hA\mathbf{u}_h(t_k)$$

$$[I + hA]\mathbf{u}_h(t_k) = \mathbf{u}(t_{k-1})$$

$$\mathbf{u}_h(t_k) = [I + hA]^{-1}\mathbf{u}_{t_{k-1}} = [I + hA]^{-k}\mathbf{u}_0$$

$$\mathbf{u}_h(t_k) = [I + hA]^{-k}\mathbf{u}_0 = V[I + h\Lambda]^{-k}V^{-1}\mathbf{u}_0 = \sum_{j=1}^n b_j(0) \frac{1}{(1 + h\lambda_j)^k} \mathbf{v}_j$$

Hence, for  $\lambda_j > 0$ , the Backward-Euler method is stable for any  $h > 0$

# Unconditional stability of trapezoidal method

Trapezoidal method with  $\Delta t = h$  produces iterations  $k = 0, 1, \dots$ :

$$\mathbf{u}_h(t_k) = \mathbf{u}_h(t_{k-1}) - \frac{1}{2}hA[\mathbf{u}_h(t_{k-1}) + \mathbf{u}_h(t_k)]$$

$$\left[I + \frac{h}{2}A\right]\mathbf{u}_h(t_k) = \left[I - \frac{h}{2}A\right]\mathbf{u}(t_{k-1})$$

$$\mathbf{u}_h(t_k) = \left[I + \frac{h}{2}A\right]^{-1} \left[I - \frac{h}{2}A\right] \mathbf{u}_{t_{k-1}} = \left[I - \frac{h}{2}A\right]^k \left[I + \frac{h}{2}A\right]^{-k} \mathbf{u}_0$$

$$\mathbf{u}_h(t_k) = \sum_{j=1}^n b_j(0) \left(\frac{1 - \lambda_j h/2}{1 + \lambda_j h/2}\right)^k \mathbf{v}_j$$

With  $\lambda_j > 0$  trapezoidal method is stable for any  $h > 0$

## Solving the diffusion equation

# Diffusion equation in 2D

$$\frac{\partial u}{\partial t} - \Delta u = f, \quad (x, y) \in \Omega, \quad 0 < t \leq T$$

$$u(x, y, t) = g(x, y, t), \quad (x, y) \in \partial\Omega, \quad 0 < t \leq T$$

$$u(x, y, 0) = u_0(x, y), \quad (x, y) \in \Omega, \quad t = 0$$

Sometimes it is written as  $(x, y, t) \in \Omega \times (0, T]$

# Time integration: Forward-Euler method

Although  $u(x, y, t)$  is a function rather than a vector, all previously explained time integration methods can still be applied. With the time step  $h$  the Forward-Euler time iterations are

$$\begin{aligned}\frac{\partial u(x, y, t)}{\partial t} &= \Delta u(x, y, t) + f(x, y, t) \\ u(x, y, t_{k+1}) &= u(x, y, t_k) + h [\Delta u(x, y, t_k) + f(x, y, t_k)] \\ u^{k+1}(x, y) &= u^k(x, y) + h [\Delta u^k(x, y) + f^k(x, y)] \\ k &= 0, 1, \dots; \quad u(x, y, t_0) = u_0(x, y)\end{aligned}$$

The time-dependent boundary condition  $u(x, y, t_k) = g(x, y, t_k)$ ,  $(x, y) \in \partial\Omega$  must also be enforced at each  $t_k$ .

# Time integration: Backward-Euler method

The Backward-Euler time iterations are

$$\frac{\partial u(x, y, t)}{\partial t} = \Delta u(x, y, t) + f(x, y, t)$$

$$u(x, y, t_{k+1}) = u(x, y, t_k) + h [\Delta u(x, y, t_{k+1}) + f(x, y, t_{k+1})]$$

$$u^{k+1}(x, y) - h \Delta u^{k+1}(x, y) = u^k(x, y) + h f^{k+1}(x, y)$$

$$k = 0, 1, \dots; \quad u(x, y, t_0) = u_0(x, y)$$

At the  $(k + 1)$ -st iteration:

- $u^k(x, y)$  and  $f^{k+1}(x, y)$  are known functions of  $(x, y) \in \Omega$
- $u^{k+1}(x, y)$  is an unknown function

# Space discretization: FDM or FVM

- In both FDM and FVM, introduce spatial grid and the solution vector  $\mathbf{u}^k \in \mathbb{R}^n$ , related to the grid values  $u(x_i, y_j, t_k)$ . Discretize the Laplacian operator  $-\Delta \rightarrow A \in \mathbb{R}^{n \times n}$ , moving Dirichlet boundary conditions to the right-hand side  $\mathbf{f}^k$  as usual
- For example, Backward-Euler iteration becomes

$$u^{k+1}(x, y) - h\Delta u^{k+1}(x, y) = u^k(x, y) + hf^{k+1}(x, y)$$

$$\mathbf{u}^{k+1} + hA\mathbf{u}^{k+1} = \mathbf{u}^k + h\mathbf{f}^{k+1}$$

$$[I + hA] \mathbf{u}^{k+1} = \mathbf{u}^k + h\mathbf{f}^{k+1}$$

$$k = 0, 1, \dots; \quad \mathbf{u}^0 = \mathbf{u}_0$$

- To find the current grid values  $\mathbf{u}^{k+1}$  in backward Euler we need to solve a linear system. This is computationally equivalent to solving one steady-state problem at each iteration

## Solving the diffusion equation with FEM



# Diffusion equation in 2D

$$\frac{\partial u}{\partial t} - \Delta u = f, \quad (x, y) \in \Omega, \quad 0 < t \leq T$$

$$u(x, y, t) = g(x, y, t), \quad (x, y) \in \partial\Omega, \quad 0 < t \leq T$$

$$u(x, y, 0) = u_0(x, y), \quad (x, y) \in \Omega, \quad t = 0$$

Sometimes it is written as  $(x, y, t) \in \Omega \times (0, T]$

# FEM with Forward Euler method

$$\begin{aligned}u^{k+1}(\mathbf{x}) &= u^k(\mathbf{x}) + h \left[ \Delta u^k(\mathbf{x}) + f^k(\mathbf{x}) \right] \\ \int_{\Omega} u^{k+1} v \, d\Omega &= \int_{\Omega} \left[ u^k + h \Delta u^k + h f^k \right] v \, d\Omega \\ \int_{\Omega} u^{k+1} v \, d\Omega &= \int_{\Omega} \left( u^k v - h \nabla u^k \cdot \nabla v + h f^k v \right) d\Omega \\ a(u^{k+1}, v) &= L_k(v)\end{aligned}$$

- $u^k$  is known, solve for  $u^{k+1}$
- System matrix is very simple in this case (mass matrix)
- Right-hand side changes every  $k$ -step, must be re-assembled
- A very simple linear system must be solved at every  $k$ -step

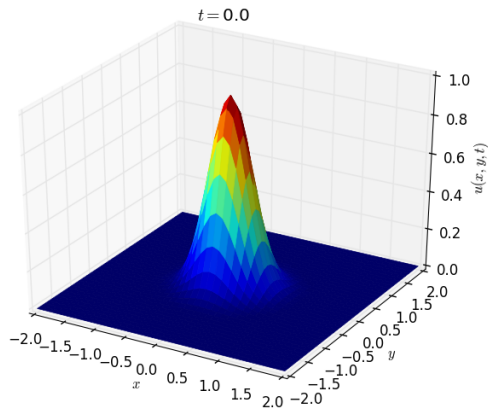
# FEM with Backward-Euler method

:

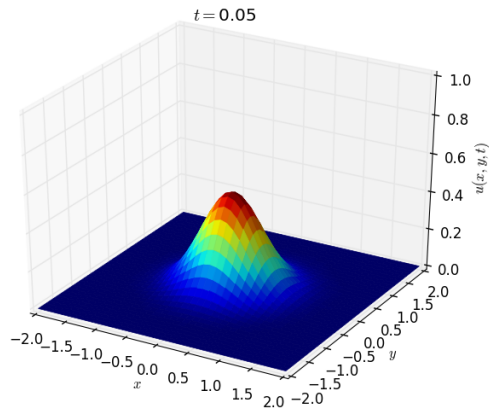
$$\begin{aligned}u^{k+1}(\mathbf{x}) - h\Delta u^{k+1}(\mathbf{x}) &= u^k(\mathbf{x}) + hf^{k+1}(\mathbf{x}) \\ \int_{\Omega} \left[ u^{k+1} - h\Delta u^{k+1} \right] v \, d\Omega &= \int_{\Omega} \left[ u^k + hf^{k+1} \right] v \, d\Omega \\ \int_{\Omega} \left( u^{k+1} v + h\nabla u^{k+1} \cdot \nabla v \right) d\Omega &= \int_{\Omega} \left( u^k v + hf^{k+1} v \right) d\Omega \\ a(u^{k+1}, v) &= L_k(v)\end{aligned}$$

- $u^k$  is known, solve for  $u^{k+1}$
- System matrix is more complicated in this case, but its assembly may be performed only once
- Right-hand side changes in every  $k$ -step, must be re-assembled
- A linear system must be solved at every  $k$ -step

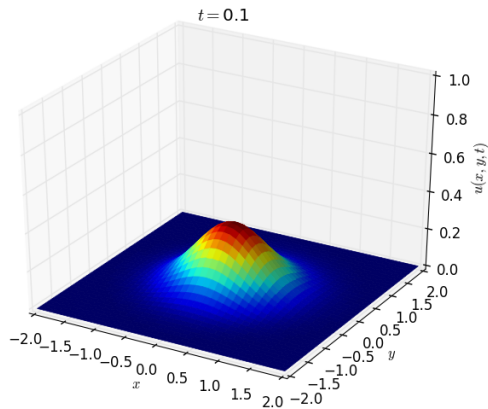
# Diffusion: $t = 0.0$ (initial condition)



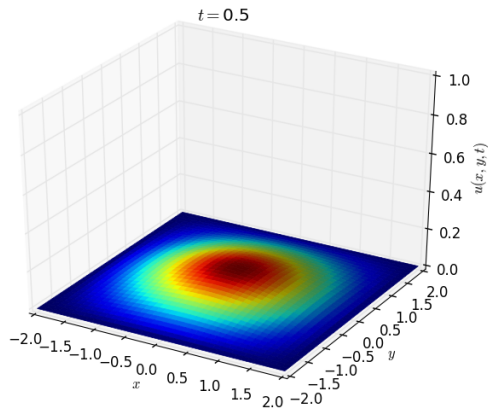
# Diffusion: $t = 0.05$



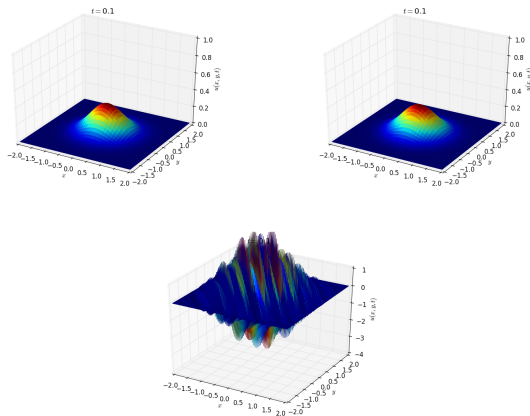
# Diffusion: $t = 0.1$



# Diffusion: $t = 0.5$



# Backward vs Forward Euler: $t = 0.1$



Left: backward Euler with  $\Delta t = 0.01$  (10 steps)

Middle and right: forward Euler with  $\Delta t = 0.0007$  (150 steps) and  $\Delta t = 0.00095$  (105 steps)